
CS771: Introduction to Machine Learning

Assignment 2 Report

Neeraj Kumar (230692) Shreya Pazare (230986) Shubhangam Raj (231000)
Kavita Kumari (230552) Arkajyoti Santra (230194)
Team Deep_Coders (Group 27)
Indian Institute of Technology Kanpur

1 Problem 2.1: Bit-in-the-Middle Attack

1.1 Part 1: Alternating Optimization for the Simple Latent Model (20 marks)

We want $(\mathbf{w}, b) \in \mathbb{R}^{17} \times \mathbb{R}$ that maximizes the likelihood of the observed responses r_i , treating the unknown middle bit $z_i \in \{0, 1\}$ as a latent variable with a uniform prior $P[z_i \mid \mathbf{c}_i, \mathbf{w}, b] = 0.5$. Taking negative logs and applying the winner-take-all approximation to the sum over z_i gives

$$\min_{(\mathbf{w}, b), \{z_i\}} L(\mathbf{w}, b, \mathbf{z}) = - \sum_{i=1}^{8000} \ln P[r_i, z_i \mid \mathbf{c}_i, \mathbf{w}, b]. \quad (1)$$

Factoring the joint via the chain rule,

$$P[r_i, z_i \mid \mathbf{c}_i, \mathbf{w}, b] = P[r_i \mid z_i, \mathbf{c}_i, \mathbf{w}, b] \cdot P[z_i \mid \mathbf{c}_i, \mathbf{w}, b] = \sigma((2r_i - 1)(\mathbf{w}^\top \phi(\mathbb{I}(\mathbf{c}_i, z_i)) + b)) \cdot 0.5,$$

so

$$L(\mathbf{w}, b, \mathbf{z}) = - \sum_{i=1}^{8000} \ln \sigma((2r_i - 1)(\mathbf{w}^\top \phi(\mathbb{I}(\mathbf{c}_i, z_i)) + b)) - 8000 \ln(0.5). \quad (2)$$

The additive constant $-8000 \ln(0.5)$ does not affect the arg min and is dropped. What remains is a joint optimization over continuous parameters $(\mathbf{w}, b)$ and 8000 discrete variables z_i , solved by **alternating coordinate descent**.

Step 1: Latent update (z -step). Fix $(\mathbf{w}, b)$. Since $-\ln \sigma(\cdot)$ is monotonic, maximizing the per-example log-likelihood over $z_i \in \{0, 1\}$ reduces to a simple margin comparison:

$$z_i \leftarrow \begin{cases} 1 & \text{if } (2r_i - 1)(\mathbf{w}^\top \phi(\mathbb{I}(\mathbf{c}_i, 1)) + b) > (2r_i - 1)(\mathbf{w}^\top \phi(\mathbb{I}(\mathbf{c}_i, 0)) + b) \\ 0 & \text{otherwise.} \end{cases} \quad (3)$$

Step 2: Model update ($(\mathbf{w}, b)$ -step). Fix $\mathbf{z}$. The problem reduces to a standard convex logistic regression over the 17-bit embedded features $\phi(\mathbb{I}(\mathbf{c}_i, z_i))$ with labels r_i :

$$\min_{(\mathbf{w}, b)} - \sum_{i=1}^{8000} \ln \sigma((2r_i - 1)(\mathbf{w}^\top \phi(\mathbb{I}(\mathbf{c}_i, z_i)) + b)). \quad (4)$$

Alternate Steps 1 and 2 until $\mathbf{z}$ stops changing (guaranteed to terminate since the objective is non-increasing each round and there are finitely many discrete configurations of $\mathbf{z}$). Because the objective is non-convex, run multiple random restarts of $\mathbf{z}^{(0)}$ and keep the run with the lowest final loss.

This alternating optimization algorithm is implemented as `my_latent()` in `submit.py`. The core loop directly mirrors the two-step derivation above: the $(\mathbf{w}, b)$ -step is a standard logistic regression call (features $\phi(\mathbb{I}(\mathbf{c}_i, z_i))$, labels r_i), and the z -step compares the two candidate margins for $z_i = 0$ and $z_i = 1$. The full procedure is repeated from three different initializations of $\mathbf{z}^{(0)}$ ($\mathbf{z}^{(0)} = \mathbf{y}$, a random 0/1 vector, and $\mathbf{z}^{(0)} = 1 - \mathbf{y}$), and the run with the highest training log-likelihood is returned.

1.2 Part 3: Alternating Optimization for the Updated Coupled Model (30 marks)

It is revealed that the middle bits z_i are not arbitrary: they are themselves the responses of a hidden 16-bit arbiter PUF $(\mathbf{u}, a) \in \mathbb{R}^{16} \times \mathbb{R}$ applied to $\mathbf{c}_i$. The updated likelihood and latent prior are

$$P[r_i \mid z_i, \mathbf{c}_i, \mathbf{w}, b, \mathbf{u}, a] = \sigma((2r_i - 1)(\mathbf{w}^\top \phi(\mathbb{I}(\mathbf{c}_i, z_i)) + b)) \quad (5)$$

$$P[z_i \mid \mathbf{c}_i, \mathbf{w}, b, \mathbf{u}, a] = \sigma((2z_i - 1)(\mathbf{u}^\top \phi(\mathbf{c}_i) + a)). \quad (6)$$

The MLE problem (max product form) is equivalent, after taking negative logs, introducing the latent variable, and applying winner-take-all, to

$$\min_{(\mathbf{w}, b), (\mathbf{u}, a), \{z_i\}} L_{\text{upd}} = - \sum_{i=1}^{8000} \left[\ln P[r_i \mid z_i, \mathbf{c}_i, \mathbf{w}, b] + \ln P[z_i \mid \mathbf{c}_i, \mathbf{u}, a] \right], \quad (7)$$

which expands to a sum of two coupled sigmoid log-likelihood terms per example. This is solved by alternating over three blocks.

Step 1: Latent update (z -step). Fix $(\mathbf{w}, b, \mathbf{u}, a)$; pick the z that maximizes the sum of both terms:

$$z_i \leftarrow \arg \max_{z \in \{0,1\}} \left[\ln \sigma((2r_i - 1)(\mathbf{w}^\top \phi(\mathbb{I}(\mathbf{c}_i, z)) + b)) + \ln \sigma((2z - 1)(\mathbf{u}^\top \phi(\mathbf{c}_i) + a)) \right]. \quad (8)$$

Step 2: 17-bit model update ($(\mathbf{w}, b)$ -step). Fix $\mathbf{z}$; isolates the first term, again a standard logistic regression on $\phi(\mathbb{I}(\mathbf{c}_i, z_i)) \rightarrow r_i$:

$$\min_{(\mathbf{w}, b)} - \sum_{i=1}^{8000} \ln \sigma((2r_i - 1)(\mathbf{w}^\top \phi(\mathbb{I}(\mathbf{c}_i, z_i)) + b)). \quad (9)$$

Step 3: 16-bit model update ($(\mathbf{u}, a)$ -step). Fix $\mathbf{z}$; isolates the second term, a logistic regression on $\phi(\mathbf{c}_i) \rightarrow z_i$ (i.e. $\mathbf{z}$ acts as the training label for the hidden 16-bit PUF):

$$\min_{(\mathbf{u}, a)} - \sum_{i=1}^{8000} \ln \sigma((2z_i - 1)(\mathbf{u}^\top \phi(\mathbf{c}_i) + a)). \quad (10)$$

Iterate Steps 1–3 to convergence, with multiple restarts of $\mathbf{z}^{(0)}$ as in Part 1, keeping the run with the best joint objective.

This algorithm is implemented as `my_latent_updated()` in `submit.py`. Each round alternates: (i) fit $(\mathbf{w}, b)$ via logistic regression on $\phi(\mathbb{I}(X, \mathbf{z})) \rightarrow \mathbf{y}$, (ii) fit $(\mathbf{u}, a)$ via logistic regression on $\phi(X) \rightarrow \mathbf{z}$, and (iii) recompute $\mathbf{z}$ by comparing the combined log-sigmoid scores from Step 1 above. As in `my_latent()`, three restarts of $\mathbf{z}^{(0)}$ are tried and the run with the highest joint objective is returned.

1.3 Part 5: Empirical Comparison of `my_latent()` and `my_latent_updated()` (20 marks)

Both functions were run on the actual training data, `public_trn.txt` (8000 rows, first 16 columns = challenge c_i , last column = response r_i). Because both functions use unseeded `np.random.randint` for one of their three restarts, they were run for 12 independent global NumPy seeds to characterize variability rather than relying on a single run.

Table 1: Empirical comparison of `my_latent()` and `my_latent_updated()` across 12 random seeds on the 8000 public training CRPs.

Quantity	Mean	Std. dev.	Range
cosine similarity($\mathbf{w}, \hat{\mathbf{w}}$)	0.014	0.330	−0.606 to 0.579
$ b - \hat{b} $	0.387	0.240	0.075 to 0.834
latent alignment fraction ($2z_i - 1 = \text{sign}(\hat{\mathbf{u}}^\top \phi(\mathbf{c}_i) + \hat{a})$)	0.468	0.098	0.247 to 0.621
train accuracy on r (<code>my_latent</code>)	0.9998	0.0001	0.9998 to 1.0000
train accuracy on r (<code>my_latent_updated</code>)	0.789	0.074	0.735 to 0.999

(a) **Did $\mathbf{w} \approx \hat{\mathbf{w}}$?** No, not reliably. Mean cosine similarity is only 0.014 (effectively uncorrelated), with high variance (std 0.33) and sign flips across restarts. The two weight vectors do not consistently point in a similar direction in the 17-bit embedded space.

(b) **Did $b \approx \hat{b}$?** No. The bias terms differ by 0.39 on average, with differences up to 0.83, which is not negligible relative to the typical bias magnitudes observed (roughly −0.4 to −1.1).

(c) **Latent alignment fraction.** Averaged 46.8% of the 8000 training examples have $2z_i - 1 = \text{sign}(\hat{\mathbf{u}}^\top \phi(\mathbf{c}_i) + \hat{a})$. This is statistically indistinguishable from the 50% expected by chance for a binary variable.

Why the two algorithms disagree. The additional diagnostic of train-set response accuracy explains the gap. `my_latent()`, free to choose any z_i independently, fits the responses almost perfectly (99.98% accuracy). With an unconstrained middle bit, the 17-bit feature map has enough capacity to memorize nearly arbitrary response patterns. `my_latent_updated()`, which additionally requires $\mathbf{z}$ to be explainable by a 16-bit arbiter PUF ($\mathbf{u}, a$), achieves markedly lower accuracy ($\approx 79\%$ on average, and as low as 73.5% in some runs). If the true generating process for the middle bits really were a clean, linearly separable 16-bit arbiter PUF, imposing that structure should cost little accuracy and the two algorithms should converge to similar $(\mathbf{w}, b)$ and highly aligned $\mathbf{z}$. Instead, the constraint measurably hurts the fit, and the two algorithms select largely uncorrelated latent-bit assignments. This is consistent with the alternating, non-convex, winner-take-all optimization landscape having many local optima of comparable objective value the two problems (7000+ free binary parameters vs. an objective additionally coupled through a 16-dimensional linear prior) are related but distinct, and are not guaranteed to converge to the same solution. In practice, they do not tend to converge to the same solution.

Note on reproducibility: both functions rely on unseeded randomness for one of their three restarts, so exact values will vary between runs; the ranges above characterize that variability across 12 independent seeds rather than reflecting a single run.